23127709 - Lê Thị Ánh Nhung 16/10/2026

Sơ đồ luồng kiến trúc CI Pipeline

- Git push Event — (kích hoạt pipeline khi push/PR vào main)
- ↓
- Checkout Code — (actions/checkout@v4 - Tải mã nguồn về Runner)
- ↓
- Setup Python 3.11 — (actions/setup-python@v5 - khởi tạo môi trường)
- ↓
- Install Dependencies — (pip install -r requirements.txt Cthim phiên bản)
- ↓
- Static Analysis — (pylint 8 - kiểm tra cú pháp & Linter)
- ↓
- Execute tests & Quality Gate (>= 80%) — (Coverage run & report --fail-under = 80) -> [pass] Upload Artifacts / [FAIL] Stop Build

23127709 - Lê Thị Ánh Nhung 16/10/2026

Bản mô tả thiết kế kịch bản Test Case Mocking

Tên Test Case	Hàm được gọi	Đối tượng & Giá trị Mocking	Kết quả kỳ vọng (Assert)
Test_promo_online_valid	apply_promo_code	net.test_status = "online" os_session = 15.code = "Giảm 10"	Trả về 90000.0 (giảm 10%)
Test_promo_offline_connection_error	apply_promo_code	net.test_status = "offline" code = "Giảm 10"	Ném ra ngoại lệ ConnectionError
Test_promo_new_os_valid	apply_promo_code	net.test_status = "online" os_session = 15.code = "New_os_only"	Trả về 80000.0 (giảm 20%)
Test_promo_old_os_session_value_error	apply_promo_code	net.test_status = "online" os_session = 14.code = "New_os_only"	Ném ra ngoại lệ ValueError

23127709 - Lê Thị Ánh Nhung 16/10/2026

Test_get_restaurants_in_range	get_nearby_restaurants	@patch: get_current_location.return_value = "10.0, 105.0"	Trả về "Nhà hàng Phở Lý Quốc Sư", "Bún chả Hương Liên"
Test_get_restaurants_out_of_range	get_nearby_restaurants	@patch: get_current_location.return_value = ("30.0, 110.0")	Trả về danh sách rỗng []
Test_get_restaurants_gps_lost_error	get_nearby_restaurants	@patch: get_current_location.side_effect = GPSSignalLostError()	Trả về chuỗi "Không thể xác định vị trí

23127709 - Lê Thị Ánh Nhung 16/10/2026

Câu 2.1: Tại sao bước chạy Linter (flake8) phải đặt trước bước chạy test trong đường ống CI?

1. Nguyên lý Thất bại sớm (Fail-Fast): Linter giúp phát hiện ngay lập tức các lỗi cú pháp, lỗi thụt lề đơn giản... rất nhanh mà không tốn tài nguyên thực thi mã nguồn. Nếu mã bị lỗi cú pháp, việc dùng CI ngay lập tức giúp tiết kiệm thời gian, CPU/RAM của Server Runner và không mất thời gian chạy bộ unit test vô ích

Câu 2.2: Tham số --fail-under=80 của lệnh coverage report hoạt động theo nguyên lý nào trong hệ điều hành Linux để báo hiệu cho GitHub Actions dừng tiến trình build?

Trong Linux / Unix, mọi tiến trình sau khi kết thúc đều trả về 1 mã thoát (Exit Code). Mã 0 tượng trưng cho thành công (Success), còn mã khác 0 (1 đến 225) báo coverage report --fail-under=80 sẽ trả về Exit Code = 1. Runner của Github Action kiểm tra Exit Code sau mỗi step, thấy khác 0, nó sẽ ngay lập tức ngắt tiến trình build, đánh dấu step đó màu đỏ (FAILED) và thông báo dừng

23127O9 - Lê Thị Ánh Nhung 16/10/2026

Câu 2.3 Tham số -m trong lệnh coverage report -m hiển thị thêm thông tin gì và giúp ích gì cho kỹ sư kiểm thử khi rà soát độ bao phủ mã nguồn?

Tham số -m hiển thị thêm cột missing liệt kê chính xác số dòng và phạm vi các dòng lệnh chưa được thực thi trong bộ test. Thông tin này giúp kỹ sư kiểm thử (SET) định vị chính xác nhánh logic hoặc hàm nào bị bỏ sót để viết bổ sung test case tập trung vào đúng các dòng mà đồ nhằm đưa coverage việt mới Quality Gate

Câu 3.1 Phân biệt giữa khái niệm Flaky Test và lỗi do sai lệch môi trường phụ thuộc

- Flaky test: là test case trả về kết quả không ổn định trên cùng 1 môi trường mã nguồn mà không hề có sự thay đổi code nào. Nguyên nhân thường do bất đồng bộ, phụ thuộc vào thời gian thực, hoặc do không dọn dẹp Static giữa các test case
- Lỗi do sai lệch môi trường phụ thuộc: là lỗi

Mặt trang vở một tương lai TTBOOK

23127O9 - Lê Thị Ánh Nhung 16/10/2026

xảy ra khi test case chạy PASS ở máy Local nhưng FAIL trên CI Runner (hoặc ngược lại) Nguyên nhân do sự khác biệt về phiên bản Python, OS (Window vs Linux) thiếu thư viện hoặc khác biệt Time zone (locale)

Câu 3.2 Tại sao trên máy chủ CI bước cài đặt thư viện từ requirements.txt (thêm flag Pinning) trong requirements.txt?

Cài flag phiên bản (Sử dụng toán tử == như requests == 2.31.0) giúp đảm bảo tính đồng nhất tuyệt đối giữa môi trường phát triển local và máy chủ CI. Nếu không ghim phiên bản, mỗi lần CI chạy nó sẽ tự động tải bản phiên bản mới nhất của thư viện, khi thư viện bên thứ 3 cập nhật có breaking changes hoặc bị deprecated, pineline CI có thể sụp đổ đột ngột dù mã nguồn dự án không thay đổi

Mặt trang vở một tương lai TTBOOK

23127O9 - Lê Thị Ánh Nhung 16/10/2026

Câu 3.3 Trình bày quy trình 6 bước chuẩn để nâng cấp 1 thư viện lên phiên bản mới an toàn qua Pull Request mà không làm gián đoạn nhánh main

1. Bắt đầu (Tạo nhánh mới): Tạo nhánh tính năng mới từ main (VD: git checkout -b update-requests-lib)
2. Cập nhật file requirements.txt: Sửa phiên bản thư viện cần nâng cấp trong file requirements.txt trên nhánh mới
3. Kiểm thử cục bộ (local testing) Chạy pip install -r requirements.txt và thực thi toàn bộ bộ test suite trên máy local để bảo đảm bảo không bị lỗi
4. Tạo pull Request (PR): push nhánh lên github và tạo PR gộp vào nhánh main
5. Xác nhận tự động qua CI pipeline: Kiểm tra Github Actions runner tự động kích hoạt job test test trên PR đảm bảo tất cả linter, Unit Test và Quality đều trả về tích xanh (PASS)

Mặt trang vở một tương lai TTBOOK

23127O9 - Lê Thị Ánh Nhung 16/10/2026

6. Review và merge: mời đồng nghiệp review code, phê duyệt PR và tiến hành gộp merge an toàn vào nhánh main

Mặt trang vở một tương lai TTBOOK